



Contactel

Rapport Technique — Application Cordova

HTML5 · CSS3 · JavaScript + jQuery · Gestion de contacts — ESP/UCAD 2026



UNIVERSITÉ CHEIKH ANTA DIOP DE DAKAR
École Supérieure Polytechnique — ESP

RAPPORT TECHNIQUE

Contactel

Application Mobile de Gestion de Contacts — Apache Cordova

Documentation complète — HTML · CSS · JavaScript + jQuery

CRUD complet, recherche, filtres par groupe, modal d'édition — expliqués étape par étape

5 sections HTML
documentées

CSS bleu thème sombre

15 fonctions JS
documentées

CRUD complet +
recherche

Auteur

Salif Biaye

Fondateur / CEO — DIC3 Informatique & Télécommunications

Application :	Contactel
Technologie :	Apache Cordova · HTML5 · CSS3 · JavaScript + jQuery 3.7
Fonctionnalités :	CRUD contacts, recherche, filtres groupe, avatar initiales, modal
Module :	Développement Mobile — DIC3 Informatique & Télécommunications
Université :	UCAD — École Supérieure Polytechnique (ESP)
Auteur :	Salif Biaye — Édition Mai 2026

Sommaire

N°	Section	Contenu
1	Vue d'ensemble	Objectif, CRUD, tech stack
2	Structure HTML	Header, search, filtres, liste, modal, confirm
3	Feuille de style CSS	Palette bleue, avatars, modal bottom sheet
4	Logique JavaScript	15 fonctions CRUD, filtres, modal, recherche
5	Captures d'écran	Liste, modal, recherche, confirmation
6	Synthèse	Bilan technique et compétences

SECTION 1

Vue d'ensemble

Objectif, CRUD contacts, pile technologique

1.1 Qu'est-ce que Contactel ?

Contactel est une application mobile de gestion de contacts développée avec **Apache Cordova**. Elle permet d'ajouter, modifier, supprimer et rechercher des contacts, chacun associé à un nom, numéro de téléphone, email et groupe (Famille, Amis, Travail, Autre). Un système d'**avatars automatiques** génère les initiales du contact lorsqu'aucune photo n'est disponible.

Les contacts sont triés alphabétiquement et persistent via **localStorage**. L'interface propose une **barre de recherche** combinée à des **filtres par groupe**, un modal d'ajout/édition en **bottom sheet**, et une boîte de confirmation avant toute suppression.

1.2 Pile technologique

Technologie	Rôle	Version
Apache Cordova	Wrapper natif Android/iOS	Dernière stable
HTML5	Structure : header, search, filtres, liste, modal, confirm	Standard W3C
CSS3	Thème bleu sombre, avatars, bottom sheet, animations	Standard W3C
JavaScript (IIFE)	CRUD contacts, filtres, recherche, rendu DOM	ES6+
jQuery 3.7.1	Sélecteurs, events, escape XSS, délégation	CDN
localStorage	Persistence JSON (clé : 'contacts')	Web API native

1.3 Fonctionnalités clés

- **CRUD complet** : Create (ajouter), Read (lister), Update (modifier), Delete (supprimer).
- **Recherche temps réel** : filtre simultané nom + téléphone dès la frappe.
- **Filtres par groupe** : Tous / Famille / Amis / Travail / Autre.
- **Avatar initiales** : génération automatique des 2 premières lettres du nom.
- **Tri alphabétique** : contacts toujours triés par nom (localeCompare 'fr').
- **Confirmation suppression** : boîte de dialogue avant toute suppression.

Synthèse

- ❑ CRUD complet + recherche + filtres : application de gestion complète
- ❑ Avatar initiales : initials() calcule 2 lettres depuis le nom
- ❑ Tri : localeCompare('fr') pour un tri alphabétique correct en français
- ❑ Thème bleu foncé (#07101A → #60A5FA) avec mode clair automatique

SECTION 2

Structure HTML

Analyse des 5 sections du fichier `index.html`

2.1 App, header et barre de recherche

□ `index.html` — header + search

```
<div id="app">
  <header>
    <h1>Contactel</h1>
    <p id="summary">0 contact(s)</p> <!-- Mis à jour par renderSummary() -->
  </header>

  <!-- Barre de recherche combinée avec les filtres -->
  <div id="search-bar">
    <span class="search-icon">🔍</span>
    <input type="search" id="search-input" placeholder="Rechercher un contact...">
  </div>

  <div id="filters">
    <button class="filter-btn active" data-group="all">Tous</button>
    <button class="filter-btn" data-group="Famille">Famille</button>
    <button class="filter-btn" data-group="Amis">Amis</button>
    <button class="filter-btn" data-group="Travail">Travail</button>
    <button class="filter-btn" data-group="Autre">Autre</button>
  </div>

  <ul id="contact-list"></ul> <!-- Rempli dynamiquement -->
  <button id="btn-add">+</button> <!-- FAB fixe -->
</div>
```

2.2 Modal d'ajout/édition

□ `index.html` — modal

```
<div id="modal-overlay" class="hidden">
  <div id="modal">
    <div id="modal-header">
      <h2 id="modal-title">Nouveau contact</h2> <!-- Ou "Modifier le contact" -->
      <button id="modal-close">✕</button>
```

```

</div>

<div id="modal-body">

  <!-- Prévisualisation de l'avatar -->

  <div class="avatar-preview" id="avatar-preview">

    <span id="avatar-preview-initials"></span> <!-- Initiales JS -->

  </div>

  <div class="field"> <label for="f-name">Nom complet *</label>

  <input type="text" id="f-name" maxlength="60"> </div>

  <div class="field"> <label for="f-phone">Téléphone *</label>

  <input type="tel" id="f-phone" maxlength="20"> </div>

  <div class="field"> <label for="f-email">Email</label>

  <input type="email" id="f-email" maxlength="80"> </div>

  <div class="field"> <label for="f-group">Groupe</label>

  <select id="f-group"> options Autre/Famille/Amis/Travail </select> </div>

  <p id="form-error" class="hidden">Champs obligatoires.</p>

</div>

<div id="modal-footer">

  <button id="btn-cancel">Annuler</button>

  <button id="btn-save">Enregistrer</button>

</div>

</div>

</div>

```

Le même modal gère **ajout ET modification**. La distinction se fait via `state.editingId` : `null` = nouveau, valeur = édition. Le titre change dynamiquement : 'Nouveau contact' ou 'Modifier le contact'.

2.3 Boîte de confirmation de suppression

□ `index.html` — `confirm dialog`

```

<div id="confirm-overlay" class="hidden">

  <div id="confirm-box">

    <p id="confirm-msg">Supprimer ce contact ?</p> <!-- Nom inséré par JS -->

    <div id="confirm-actions">

      <button id="confirm-no">Non</button>

      <button id="confirm-yes">Oui, supprimer</button>

    </div>

  </div>

</div>

```

Le message de confirmation inclut le nom du contact : 'Supprimer "Saliou Diallo" ?'. Cela améliore l'UX en évitant les suppressions accidentelles.

Synthèse

- ❑ Modal unique pour ajout ET édition — `state.editingId` distingue les deux cas
- ❑ `data-group` sur les boutons filtres — JS lit cet attribut pour `setFilter()`
- ❑ `type=search` sur l'input : bouton clear (×) natif sur iOS/Android
- ❑ Boîte de confirmation avec nom du contact — protection contre les fausses suppressions

SECTION 3

Feuille de style CSS

Palette bleue, système d'avatars, bottom sheet et animations

3.1 Palette de couleurs

Élément	Hex	Usage
fond global	#07101A	Fond arrière très sombre (bleu-noir)
header gradient	#1E3A8A → #60A5FA	Dégradé en-tête
accent blue	#1D4ED8	Boutons, FAB, filtre actif
accent clair	#60A5FA	Hover, focus, éléments secondaires
texte	#E0EDF8	Texte principal (bleu très clair)
input border	#1E3A5A	Bordure des champs de formulaire
btn-edit	#60A5FA	Bouton modifier (bleu clair)
btn-delete	#F87171	Bouton supprimer (rouge clair)
badge groupe	variable	Couleur selon le groupe du contact

3.2 Système d'avatars

Chaque contact a un avatar circulaire de 48px. Si une image est disponible (img/contact.png), elle est affichée. Sinon, le JS génère les initiales et les affiche sur fond dégradé bleu.

❏ index.css — avatar contact

```
.contact-avatar {
width: 48px; height: 48px; border-radius: 50%;
overflow: hidden; flex-shrink: 0;
background: linear-gradient(135deg, #1d4ed8, #60a5fa); /* Fond initiales */
display: flex; align-items: center; justify-content: center;
font-weight: 700; color: white; font-size: 16px;
}

/* Prévisualisation dans le modal (64px) */
.avatar-preview {
width: 64px; height: 64px; border-radius: 50%;
background: linear-gradient(135deg, #1e3a8a, #60a5fa);
/* JS bascule entre img et span#initiales */
}
```



```
}

```

3.3 Animations

Animation	Durée	Déclencheur	Effet
slideIn	0.2s ease	Nouveau contact	Contact entre depuis le bas
slideUp	0.25s ease	Ouverture modal	Modal monte depuis le bas
fadeOut	0.18s ease	Overlay modal	Fondu de l'arrière-plan
zoomIn	0.2s ease	Boîte de confirm	La boîte grossit depuis 0.8×

Synthèse

- ❑ Avatar : fond dégradé bleu avec initiales JS — dégradé naturel même sans photo
- ❑ Bottom sheet modal : slideUp 0.25s — imite le comportement natif iOS/Android
- ❑ zoomIn confirm : 0.8× → 1.0× — feedback visuel fort avant suppression
- ❑ Mode clair : prefers-color-scheme — fond #eff6ff, texte #0d1628

SECTION 4

Logique JavaScript

IIFE + jQuery — 15 fonctions CRUD, filtres, recherche, modal documentées

4.1 Structure, état global et constantes

□ `index.js` – module + state + AVATAR_CLASS

```
var App = (function ($) {
  var STORAGE_KEY = "contacts";

  // Mapping groupe → classe CSS pour la couleur du badge
  var AVATAR_CLASS = {
    Famille: "g-Famille", // CSS : badge vert
    Amis: "g-Amis", // CSS : badge bleu clair
    Travail: "g-Travail", // CSS : badge orange
    Autre: "g-Autre" // CSS : badge gris
  };

  var state = {
    contacts: load(), // Tableau de contacts depuis localStorage
    group: "all", // Filtre groupe actif
    search: "", // Terme de recherche en cours
    editingId: null, // ID du contact en cours d'édition (null=création)
    deletingId: null // ID du contact en attente de suppression
  };

  }($));
```

4.2 Stockage, escape et initiales

□ `index.js` – helpers

```
// load : JSON.parse depuis localStorage, [] si absent ou corrompu
function load() {
  try { return JSON.parse(localStorage.getItem(STORAGE_KEY) || "[]"); }
  catch (_) { return []; }
}

// persist : sérialise le tableau complet à chaque modification
function persist() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(state.contacts));
}
```

```

}

// escape : prévenir les injections XSS via astuce jQuery
function escape(text) { return $("

").text(String(text)).html(); }

// initials : extraire les 2 premières initiales du nom
function initials(name) {
  var parts = name.trim().split(/\s+/);
  return parts.length >= 2
    ? parts[0][0] + parts[1][0] // "Salif Biaye" → "SB"
    : parts[0].slice(0, 2); // "Saliou" → "Sa"
}


```

4.3 Filtrage et tri — `getFiltered()`

□ `index.js` — `getFiltered`

```

function getFiltered() {
  return state.contacts
    .filter(function (c) {
      var matchGroup = state.group === "all" || c.group === state.group;
      var matchSearch = !state.search ||
        c.name.toLowerCase().includes(state.search) ||
        c.phone.includes(state.search); // Chercher aussi dans le téléphone
      return matchGroup && matchSearch; // Les deux conditions réunies
    })
    .sort(function (a, b) {
      return a.name.localeCompare(b.name, "fr"); // Tri alphabétique français
    });
}

// localeCompare("fr") : gère é, è, ê, ç, à correctement

```

4.4 Rendu HTML — `buildContactHtml()`

□ `index.js` — `buildContactHtml`

```

function buildContactHtml(c) {
  return '<li class="contact-item" data-id="' + c.id + '">' +
    '<div class="contact-avatar">' +
    '' +
    '</div>' +
    '<div class="contact-info">' +

```

```
'<div class="contact-name">' + escape(c.name) + '</div>' +
'<div class="contact-phone">' + escape(c.phone) + '</div>' +
'<span class="contact-group-badge">' + escape(c.group) + '</span>' +
'</div>' +
'<div class="contact-actions">' +
'<button class="btn-edit" data-id="' + c.id + '">...</button>' +
'<button class="btn-delete" data-id="' + c.id + '">...</button>' +
'</div>' +
'</li>';
}

// escape() sur name, phone, group : protection XSS systématique
```

4.5 Modal — openModal, closeModal, updateAvatarPreview, readForm, saveContact

□ index.js — modal functions

```
// openModal(id) : ouvre en mode édition (id≠null) ou création (id=null)
function openModal(id) {
  state.editingId = id;
  var c = id ? state.contacts.find(function (x) { return x.id === id; }) : null;
  $("#modal-title").text(c ? "Modifier le contact" : "Nouveau contact");
  // Pré-remplir le formulaire si édition
  $("#f-name").val(c ? c.name : "");
  $("#f-phone").val(c ? c.phone : "");
  // ...
  updateAvatarPreview(c ? c.name : "");
  $("#modal-overlay").removeClass("hidden");
}

// updateAvatarPreview : bascule entre image et initiales
function updateAvatarPreview(name) {
  var ini = name && name.trim() ? initials(name) : "";
  if (ini) {
    $("#avatar-preview-img").hide();
    $("#avatar-preview-initials").text(ini).show();
  } else {
    $("#avatar-preview-img").show();
    $("#avatar-preview-initials").hide();
  }
}
```

```
// saveContact : valide, crée ou met à jour, persist + render
function saveContact() {
  var data = readForm(); // { name, phone, email, group }
  if (!data.name || !data.phone) {
    $("#form-error").removeClass("hidden"); return;
  }
  if (state.editingId) {
    // Mise à jour : Object.assign sur le contact existant
    var existing = state.contacts.find(function (c) { return c.id === state.editingId; });
    if (existing) Object.assign(existing, data);
  } else {
    // Création : id = Date.now() (timestamp unique)
    state.contacts.push({ id: Date.now(), ...data });
  }
  persist(); render(); closeModal();
}
```

4.6 Suppression — askDelete, cancelDelete, confirmDelete

□ index.js — delete flow

```
// askDelete : mémorise l'ID et affiche la confirmation avec le nom
function askDelete(id) {
  state.deletingId = id;
  var c = state.contacts.find(function (x) { return x.id === id; });
  var name = c ? c.name : "ce contact";
  $("#confirm-msg").text('Supprimer "' + escape(name) + '" ?');
  $("#confirm-overlay").removeClass("hidden");
}

// cancelDelete : annuler — reset deletingId
function cancelDelete() {
  $("#confirm-overlay").addClass("hidden");
  state.deletingId = null;
}

// confirmDelete : filtrer le contact hors du tableau
function confirmDelete() {
  state.contacts = state.contacts.filter(function (c) {
    return c.id !== state.deletingId; // Garder tous sauf celui à supprimer
  });
  persist(); render(); cancelDelete();
}
```

```
}
```

Synthèse

- ❑ openModal(id) : double usage création/édition — state.editingId null ou valeur
- ❑ updateAvatarPreview : live preview des initiales pendant la saisie du nom
- ❑ saveContact : Object.assign pour mise à jour in-place sans remplacer l'objet
- ❑ askDelete → confirmDelete : 3 étapes sécurisées avec nom affiché dans la confirmation
- ❑ getFiltered : recherche combinée (nom OU téléphone) + filtre groupe + tri localeCompare

SECTION 5

Captures d'écran

Placer les images dans : C:\Users\DELL\Downloads\medy\screens

5.1 Liste des contacts

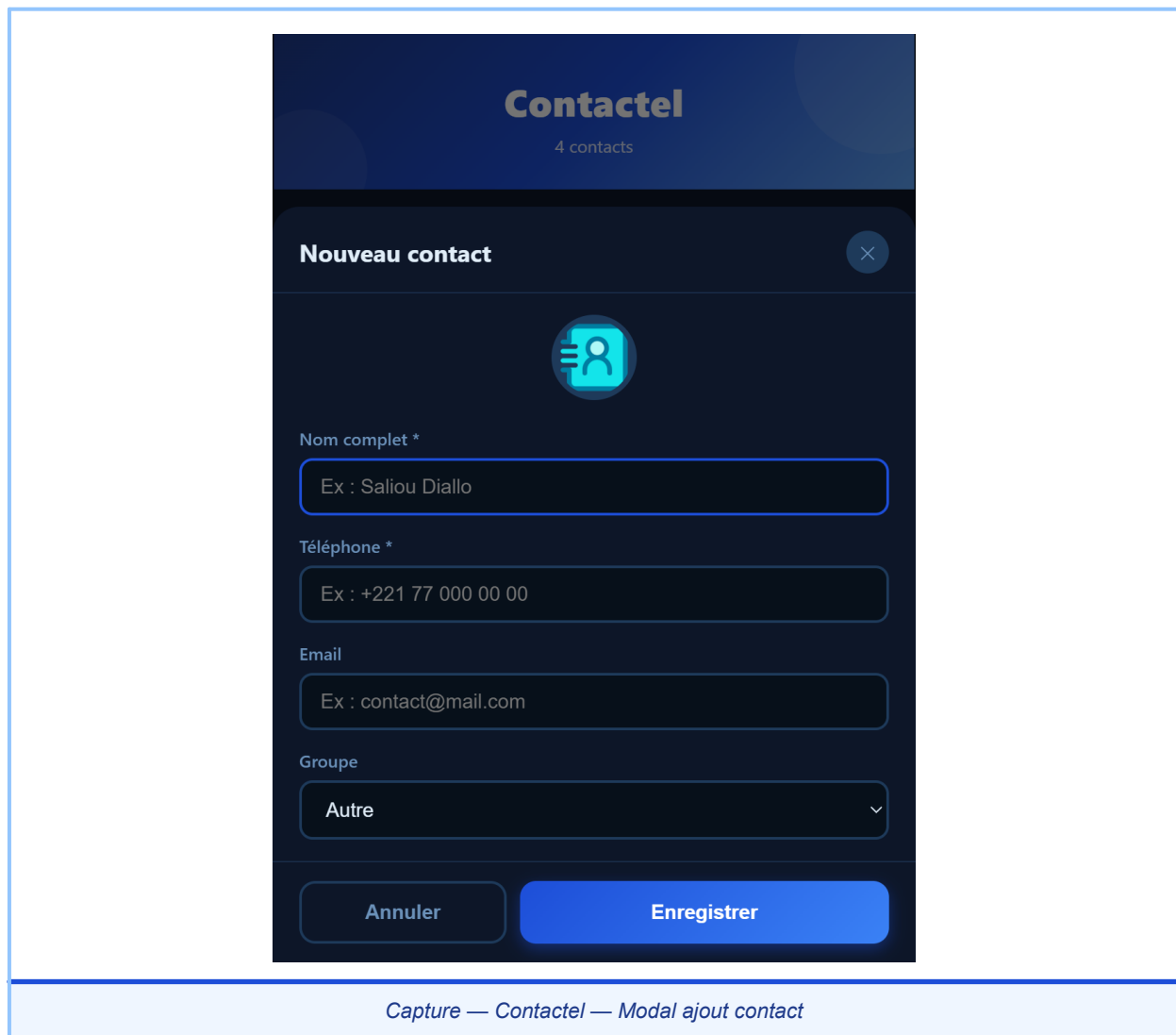
Vue principale avec liste triée, avatars, badges groupes, boutons edit/delete.



Capture — Contactel — Liste des contacts

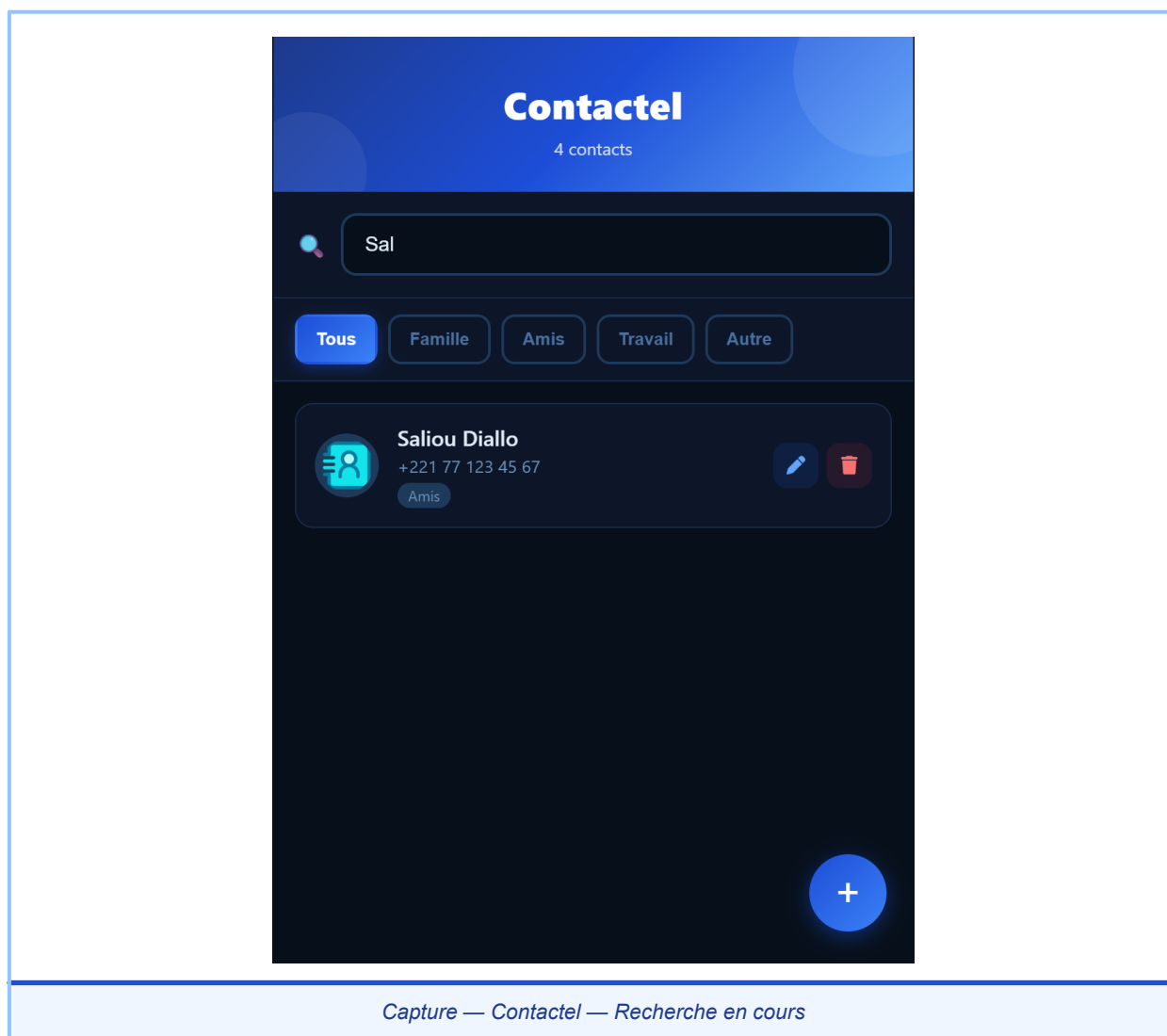
5.2 Modal d'ajout / modification

Bottom sheet avec prévisualisation de l'avatar, formulaire, validation.



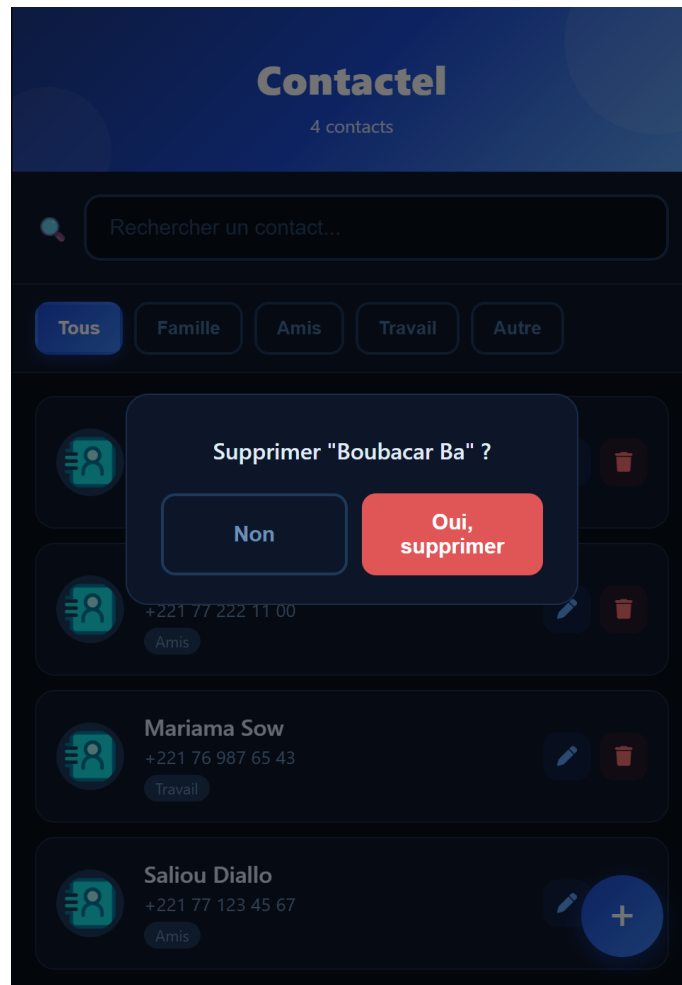
5.3 Barre de recherche active

Filtrage temps réel par nom ou téléphone.



5.4 Boîte de confirmation de suppression

Dialogue centré avec nom du contact à supprimer.



Capture — Contactel — Confirmation suppression

SECTION 6

Synthèse finale

Bilan technique et compétences démontrées

6.1 Tableau récapitulatif

Aspect	Détail	Niveau
CRUD complet	Create, Read, Update, Delete — 4 opérations fonctionnelles	☐ Complet
Recherche	Filtre temps réel sur nom ET téléphone, insensible à la casse	☐ Avancé
Avatars	initials() calcule les initiales, preview live pendant la saisie	☐ Soigné
Tri	localeCompare('fr') — alphabétique avec accents corrects	☐ Solide
UX	Bottom sheet, confirmation nominative, FAB, état vide	☐ Optimisé
Sécurité	escape() XSS sur tous les champs affichés	☐ Bon

Synthèse

- ☐ 15 fonctions : load, persist, escape, initials, getFiltered, buildContactHtml, renderList, renderSummary, render, setFilter, setSearch, openModal, closeModal, updateAvatarPreview, readForm, saveContact, askDelete, cancelDelete, confirmDelete, bindEvents, init
- ☐ Recherche combinée : nom.toLowerCase().includes() || phone.includes() — temps réel
- ☐ Object.assign : mise à jour propre des propriétés sans recréer l'objet
- ☐ Double modal : un seul panneau pour créer ET modifier — state.editingId discriminant

Contactel

Rapport Technique — Application Cordova

UCAD — École Supérieure Polytechnique (ESP)

Salif Biaye — Fondateur / CEO

Module : Développement Mobile — DIC3 — Édition Mai 2026

Document produit dans le cadre du cursus DIC3 — Usage académique uniquement